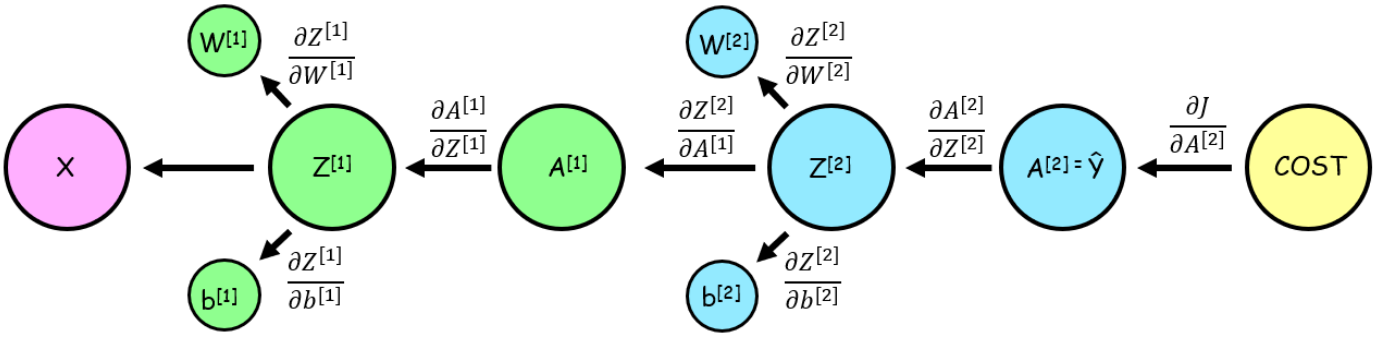


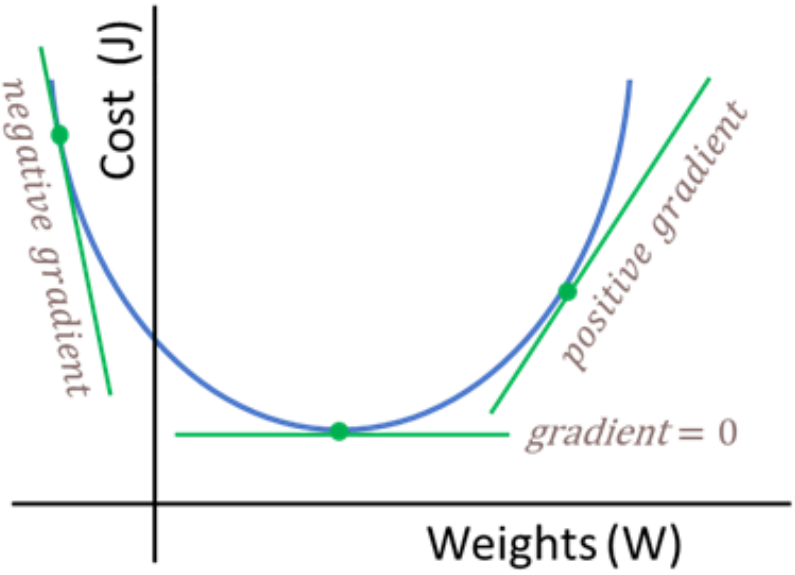
Introduction

Understanding the mathematical operations behind Neural Networks (NNs) is important for a data scientist's ability to design efficient deep learning models. In this article, the high-level calculus of a fully connected NN will be demonstrated, with focus on the backward propagation step. The article is oriented to people with basic knowledge of NNs, that seek to dive deeper into the NNs structure.

Background

The objective of the training process is to find the weights (W) and biases (b) that minimize the error. This minimization is achieved through the **gradient descent** algorithm. To begin with, the weights are randomly initialized, and an iterative process of a subtle weights change is performed until convergence.

Each iteration begins with a forward pass, that outputs the current prediction and evaluates the model's error by a cost function J (equation 6). Next, a backward pass is conducted to compute the weights gradient.



To find the best parameters that minimize the error, we use the gradient descent algorithm. The goal is to find the minimum point of the cost function (J), where the gradient is close to zero (Figure 2). The algorithm iteratively moves, step by step, in the direction of steepest descent, toward the minimum point. The step size is named the '**learning rate**', and it is a scalar that determines how much the weights change in each iteration, and therefore how quickly the NN converges. The learning rate is a hyperparameter that has to be tuned. Smaller learning rates require more training time, whereas larger learning rates result in rapid

training, but might suffer from compromised performances and instability.

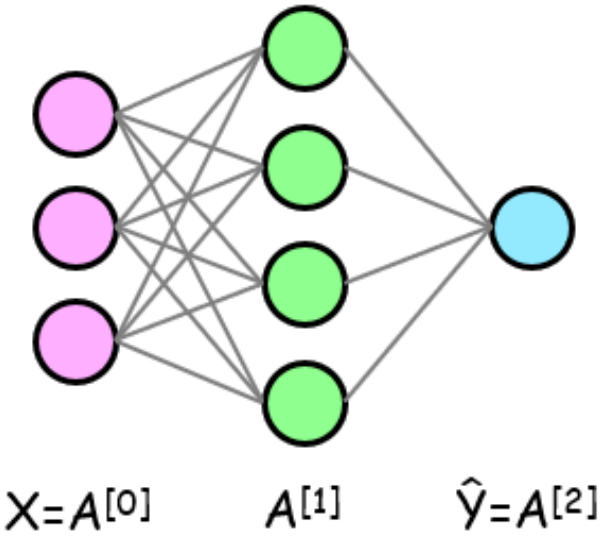
To update the weights, the gradients are multiplied by the learning rate (alpha), and the new weights are calculated by the following formula:

$$W_{new} = W_{old} - \alpha \underbrace{\frac{dJ}{dW}}_{\text{gradient}}$$

As the model iterates, the gradient gradually converges toward zero, where the error is most likely the lowest (Alternatively, the model might converge to a local optimum and present sub-optimal performances).

Describing the Net

This article will follow the structure of a two layered Neural Network, where X (also named A[0]) is the input vector, A[1] is a hidden layer, and Y-hat is the output layer. The basic architecture of the net is illustrated in the picture below (The neurons amount in each layer are irrelevant to the equations, since we are using the vectorized equations):



Forward pass

To perceive how the backward propagation is calculated, we first need to overview the forward propagation. Our net starts with a vectorized linear equation, where the layer number is indicated in square brackets.

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

Next, a non linear **activation function** (A) is added. This activation enables the net to break linearity and adjust to complex patterns in the data. Several different activation functions can be used (e.g. sigmoid, ReLU, tanh), and here we will use the sigmoid activation for our NN.

$$A^{[1]} = \sigma\left(Z^{[1]}\right) = \frac{1}{1+e^{-z}}$$

So far we computed the first layer. The second layer, like the first, is composed of a linear equation (Z[2]), followed by a sigmoid activation (A[2]). Since this is the last layer in our net, the activation result (A[2]) is the model's prediction (Y-hat).

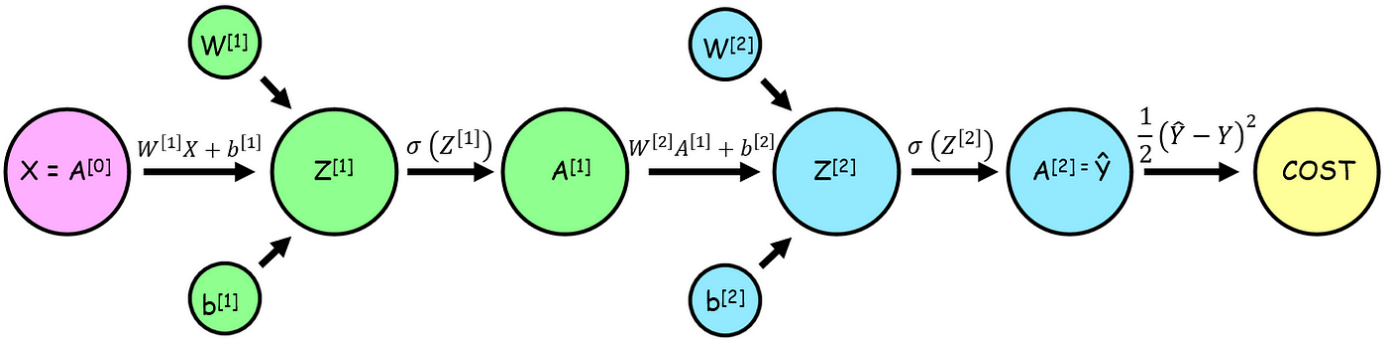
$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = \sigma\left(Z^{[2]}\right) = \frac{1}{1+e^{-z}}$$

Finally, to evaluate and minimize the error, we define a **cost function** (For further reading on activations and cost functions go to reference [1]). Here we are using the 'Mean Squared Error' (MSE) function. For simplicity, we will use Stochastic Gradient Descent (SGD) method [2], meaning that only one sample is processed in each iteration.

$$Cost\ (J) = \frac{1}{2}\left(\hat{Y} - Y\right)^2$$

We can summarize the computational graph of the forward pass

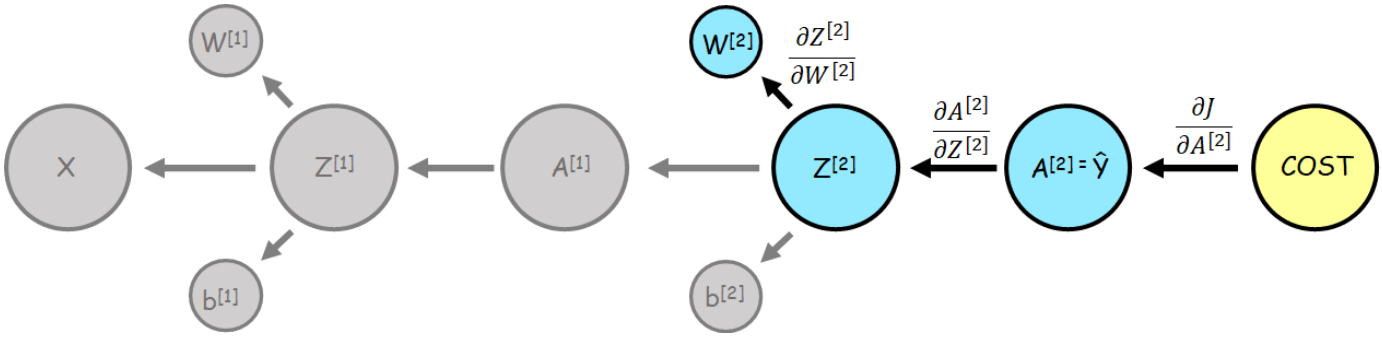


Backward pass

In order to update the weights and biases after each iteration, we need to compute the gradients. In our two layered net there are 4 parameters to be updated: W[2], b[2], W[1] and b[1], and therefore 4 gradients to be computed:

$$\frac{\partial J}{\partial W^{[2]}}, \quad \frac{\partial J}{\partial b^{[2]}}, \quad \frac{\partial J}{\partial W^{[1]}}, \quad \frac{\partial J}{\partial b^{[1]}}$$

But how are we going to find the derivatives of these composite functions? According to the '**chain rule**', we'll construct the product of the derivatives along all paths connecting the variables. Let's follow the weights of the second layer (W[2]) gradient:



From the diagram above we can clearly see that the change in the cost J with respect to W[2] is:

$$\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W^{[2]}}$$

To resolve this, we'll start by computing the partial derivative of the cost J with respect to A[2], which is also the prediction Y-hat. The original cost function is shown on the left, and the derivative on the right:

Original function:	Partial derivative:
$J = \frac{1}{2}\left(A^{[2]} - Y\right)^2$	$\frac{\partial J}{\partial A^{[2]}} = A^{[2]} - Y$

The partial derivative of the sigmoid activation in A[2], with respect to Z[2] is represented by the following (The mathematical development of the sigmoid derivative is described in ref [3]):

Original function:	Partial derivative:
$A^{[2]} = \sigma\left(Z^{[2]}\right) = \frac{1}{1+e^{-Z^{[2]}}}$	$\frac{\partial A^{[2]}}{\partial Z^{[2]}} = A^{[2]}\left(1 - A^{[2]}\right)$

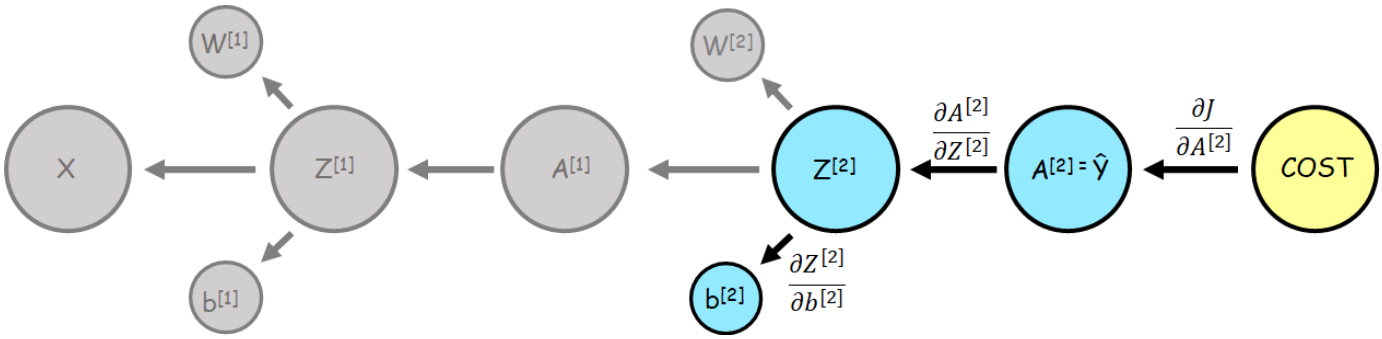
The partial derivative of Z[2] with respect to the weights W[2]:

Original function:	Partial derivative:
$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$	$\frac{\partial Z^{[2]}}{\partial W^{[2]}} = A^{[1]}$

Lets chain everything together, to compute the W[2] gradient:

$$\frac{\partial J}{\partial W^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial W^{[2]}} = (A^{[2]} - Y) \bullet A^{[2]}(1 - A^{[2]}) \bullet A^{[1]}$$

Great! Next, we will compute the b[1] gradient in a similar way. Lets follow the gradient:



$$\frac{\partial J}{\partial b^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial b^{[2]}}$$

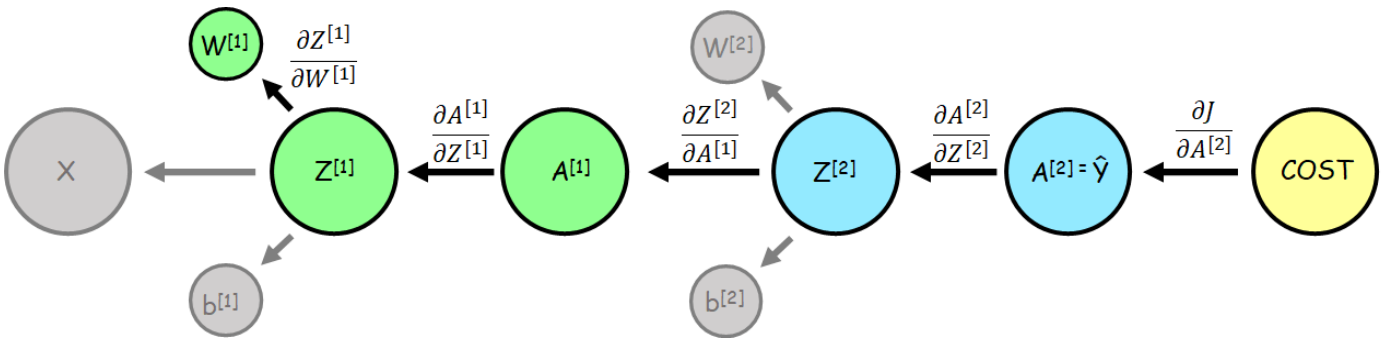
The first two parts of the b[2] gradient were already calculated above (The partial derivative of the cost J with respect to Z[2]), and the last part is equal to 1:

Original function:	Partial derivative:
$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$	$\frac{\partial Z^{[2]}}{\partial b^{[2]}} = 1$

So the overall b[2] gradient is:

$$\frac{\partial J}{\partial b^{[2]}} = \frac{\partial J}{\partial A^{[2]}} \frac{\partial A^{[2]}}{\partial Z^{[2]}} \frac{\partial Z^{[2]}}{\partial b^{[2]}} = (A^{[2]} - Y) \bullet A^{[2]}(1 - A^{[2]}) \bullet 1$$

At last, we finished computing the gradients for the second layer. The gradients for first layer are a bit longer, but we already computed parts of it. Lets follow the gradient for W[1] update:



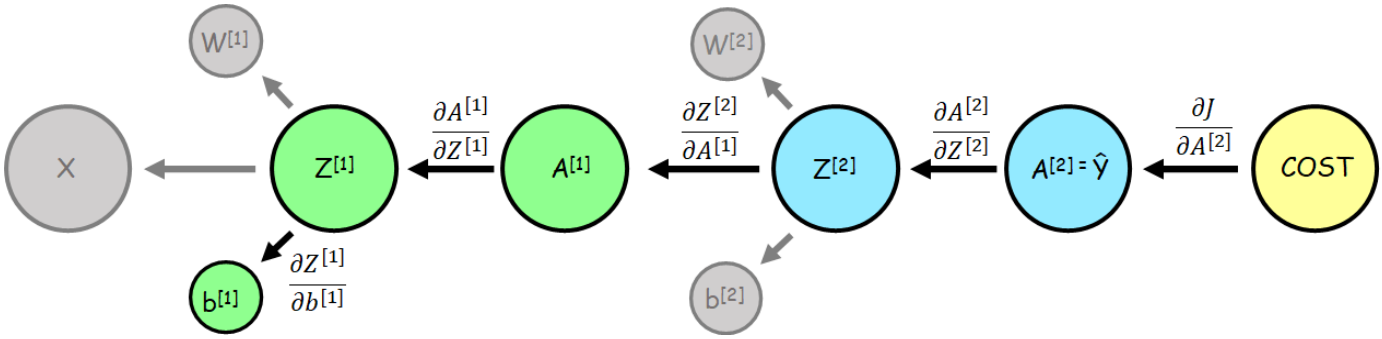
The first two parts of the gradient were previously calculated for layer 2. The partial derivative of Z[2] with respect to A[1], is W[2]:

Original function:	Partial derivative:
$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$	$\frac{\partial Z^{[2]}}{\partial A^{[1]}} = W^{[2]}$

The last two parts are calculated in the same manner as it was calculated in layer 2. Taking it together, we get:

$$\frac{\partial J}{\partial W^{[1]}} = \underbrace{\frac{\partial J}{\partial A^{[2]}}\frac{\partial A^{[2]}}{\partial Z^{[2]}}}_{\text{Previously calculated}}\frac{\partial Z^{[2]}}{\partial A^{[1]}}\frac{\partial A^{[1]}}{\partial Z^{[1]}}\frac{\partial Z^{[1]}}{\partial W^{[1]}} = (A^{[2]} - Y) \bullet A^{[2]}(1 - A^{[2]}) \bullet W^{[2]} \bullet A^{[1]}(1 - A^{[1]}) \bullet X$$

And if we follow the b[1] gradient:



We'll get:

$$\frac{\partial J}{\partial b^{[1]}} = \underbrace{\frac{\partial J}{\partial A^{[2]}}\frac{\partial A^{[2]}}{\partial Z^{[2]}}}_{\text{Previously calculated}}\frac{\partial Z^{[2]}}{\partial A^{[1]}}\frac{\partial A^{[1]}}{\partial Z^{[1]}}\frac{\partial Z^{[1]}}{\partial b^{[1]}} = (A^{[2]} - Y) \bullet A^{[2]}(1 - A^{[2]}) \bullet W^{[2]} \bullet A^{[1]}(1 - A^{[1]}) \bullet 1$$

By this part we finished computing all the gradients of the weights and biases for one iteration of our NN.

Weight update

Once computing the gradients, we can update the new parameters for the model, and iterate all over again until the model converges. Denote that alpha is the learning rate, a hyperparameter that will determine the convergence rate.

$$W_{new}^{[1]} = W_{old}^{[1]} - \alpha \frac{dJ}{dW^{[1]}} \quad | \quad b_{new}^{[1]} = b_{old}^{[1]} - \alpha \frac{dJ}{db^{[1]}}$$

$$W_{new}^{[2]} = W_{old}^{[2]} - \alpha \frac{dJ}{dW^{[2]}} \quad | \quad b_{new}^{[2]} = b_{old}^{[2]} - \alpha \frac{dJ}{db^{[2]}}$$

Exercises

Part 1: Neural Network from Scratch – Implementation Exercise

In this multi-step exercise, you will implement a simple feedforward neural network in Python. The final result will be a network that can train on data, make predictions, and export/import its structure.

🎯 Objective

Implement a complete neural network with support for training (via backpropagation), prediction, and persistence (save/load).

Suggested structure of a Neural Network in Python

🧠 **Neuron class**
Represents a single neuron in the network.
Key features:

- Stores its **weights** and **bias**
- Applies an **activation function** (default: sigmoid)
- Performs **activation** (i.e., computes output = activation(dot(inputs, weights) + bias))
- Stores its last **inputs** and **output** for use during backpropagation
- Can **update** its weights and bias during training using calculated gradients

🧱 **Layer class**
Represents a layer of neurons.
Key features:

- Contains a list of Neuron objects
- forward(inputs): Computes the output for each neuron in the layer (i.e., forward pass)
- backward(errors, learning_rate): Performs backpropagation for the layer
- Computes weight and bias updates for each neuron
- Returns the error to be passed to the previous layer (used for chain rule)

🧠 **NeuralNetwork class**
Represents the full neural network.
Key features:

- Holds a list of Layer objects
- Supports:
  - train(inputs, targets): Trains the network using forward and backward passes
  - predict(inputs): Performs inference (only forward pass)
  - dump_json() / load(): Save/load the network to/from a JSON file
  - dump_txt() / dump_initfile(): Export to .txt and .h formats

✅ Step-by-step Tasks

🔧 Step 1: Implement Neuron, Layer, NeuralNetwork

- Create Python classes for: Neuron, Layer, NeuronalNetwork
- Add the necessary fields

➡️ Step 2: Implement forward and predict

- Implement the forward pass logic:
  - forward — computes the output for the given input.
  - predict — returns the final output of the network.
- Use sigmoid (or configurable) activation function.

🔁 Step 3: Implement backward and train

- Implement the backpropagation algorithm:
  - Calculate error, deltas, and update weights.
  - Implement a training loop

📄 Step 4: Export and Import from/to JSON

- Write functions to:
  - Export a trained network to a .json file.
  - Reconstruct the network from the .json file.

📝 Step 5: Export to .txt for Human Readability

- Implement a function to export the network to a .txt file.
  - Include number of layers, neurons, weights, and biases.
  - Format the file so it is easy to read and verify manually.

⚙️ Step 6: Generate Include File (.h) with Network Constants

Generate a header file that a C programm can use to initialize the net

✍️ Optional Bonus Tasks

- Add support for different activation functions (ReLU, tanh).
- Support mini-batch training or stochastic training.
- Implement momentum or other optimization strategies.
- Create a test suite with known inputs/outputs to validate correctness.

Part 2: Neural Network – Training and Persistence Exercise

In this exercise, you will train your custom neural network implementation on real problems, export the trained network, re-import it, and verify correct behavior.

✅ Step-by-step Tasks

🔁 Step 1: Train the XOR Gate

- Network Architecture:
  - Input layer: 2 neurons
  - Hidden layer: 2 neurons
  - Output layer: 1 neuron
- Training Parameters:
  - Epochs: 20000
  - Learning rate (alpha): 0.1
- Training Data (XOR)

Input	Output
[0, 0]	[0]
[0, 1]	[1]
[1, 0]	[1]
[1, 1]	[0]

- Train your network on this dataset.
- Export the trained network to a .json file.

📥 Step 2: Load and Test Your Network

- Import the .json file from Step 1.
- Test the loaded network on the same 4 inputs.

- Verify that the predictions are similar to the outputs from the trained model (within tolerance).
- Log or print the results to confirm that export/import works correctly.

🔢 Step 3: Train a 7-Segment Decoder

- Task: Map 4-bit binary input (digits 0–9) to 7 output bits (segments A-G).
- Network Architecture:
  - Input layer: 4 neurons
  - Hidden layer: 16 neurons (can be adjusted)
  - Output layer: 7 neurons
- Training Parameters:
  - Epochs: 20000
  - Learning rate: 0.1
  - Dataset: Prepare training data that maps each digit (0–9) to its 7-segment code.

#	Input	Output
0	[0, 0, 0, 0]	[1, 1, 1, 1, 1, 1, 0]
1	[0, 0, 0, 1]	[0, 1, 1, 0, 0, 0, 0]
2	[0, 0, 1, 0]	[1, 1, 0, 1, 1, 0, 1]
3	[0, 0, 1, 1]	[1, 1, 1, 1, 0, 0, 1]
4	[0, 1, 0, 0]	[0, 1, 1, 0, 0, 1, 1]
5	[0, 1, 0, 1]	[1, 0, 1, 1, 0, 1, 1]
6	[0, 1, 1, 0]	[1, 0, 1, 1, 1, 1, 1]
7	[0, 1, 1, 1]	[1, 1, 1, 0, 0, 0, 0]
8	[1, 0, 0, 0]	[1, 1, 1, 1, 1, 1, 1]
9	[1, 0, 0, 1]	[1, 1, 1, 1, 0, 1, 1]

- Train the network.
- Export the trained model to .json.

📥 Step 4: Load and Test 7-Segment Decoder

- Load the .json from Step 3.
- Test the network on inputs [0,0,0,0] to [1,0,0,1] (i.e., digits 0 to 9).
- Compare the predicted outputs with expected 7-segment values.
- Confirm that import/export retains accuracy.

✍️ Optional Extensions

- Visualize outputs for both networks (XOR and 7-segment).
- Add accuracy metrics or thresholding to compare binary outputs.
- Try training on noisy input data (e.g., flipped bits).

Part 3: Neural Network – Comparison with TensorFlow

In this exercise, you'll reimplement the same networks from your neural network implementation but using TensorFlow in Python. This allows you to compare training performance, prediction accuracy, and general behavior.

✅ Step-by-step Tasks

🧠 Step 1: Train XOR with TensorFlow

- Use the same architecture as in your Python implementation:
  - Input: 2 neurons
  - Hidden: 2 neurons (activation: sigmoid)
  - Output: 1 neuron (activation: sigmoid)
- Dataset: Standard XOR

🔢 Step 2: Train 7-Segment Decoder with TensorFlow

- Same structure as in your C implementation:
  - Input: 4 neurons (BCD digits 0–9)
  - Hidden: e.g., 10 neurons
  - Output: 7 neurons (binary segments A–G)
- Use sigmoid activations and mean squared error loss.

📊 Step 3: Compare Results

- Compare predictions between your implementaion and the TensorFlow models:
  - On the same inputs.
  - After similar training duration (same epochs and architecture).
- Evaluate:
  - Final prediction accuracy (are outputs close to expected values?)
  - Training speed and convergence

Part 4: Neural Network – Minimal Forward Inference in C

In this exercise, you'll re-implement your neural network in C, but focus only on the forward and predict logic, without any training. Instead, the weights and biases will be loaded from the .txt file.

✅ Step-by-step Tasks

🧠 Step 1: Re-implement the Network (Forward Only)

- Define the necessary data structures:
  - Neuron with weights, bias, output
  - Layer containing neurons
  - Network containing layers
- Only implement:
  - Network and layer initialization from a .txt file (see Part 1)
  - forward() function to propagate inputs through the network
  - predict() function to get the output layer's result

➡️ Step 3: Implement and Test

Test your implementation:

- Load the .txt of a 7-seg net
- Test the network on inputs [0,0,0,0] to [1,0,0,1] (i.e., digits 0 to 9).
- Compare the predicted outputs with expected 7-segment values.
- Confirm that import retains accuracy.

Part 5: Neural Network – Forward Inference with Header Initialization

In this exercise, you'll re-implement the forward-only version of your neural network in C, but instead of loading weights from a file, you'll initialize everything using a C header file (.h) from Part 1.

✅ Step-by-step Tasks

🧠 Step 1: Reuse Forward-Only Network Structure

- Use the same network implementation as in the previous exercise:
  - Neuron, Layer, Network structs
  - forward() and predict() functions

📄 Step 2: Use the Header File for Initialization

- Use the header file (e.g., 7seg_net.h) from Part 1 containing:
  - Number of layers and sizes
  - All neuron biases and weights
  - A function to initialize the network

🧪 Step 3: Test the Network

Test your implementation:

- Use the Include file for the 7-seg net
- Test the network on inputs [0,0,0,0] to [1,0,0,1] (i.e., digits 0 to 9).
- Compare the predicted outputs with expected 7-segment values.
- Confirm that import retains accuracy.

Part 6: Neural Network on ESP32 – 7-Segment Decoder via Serial

In this final exercise, you'll port your forward-only neural network to run on an ESP32 microcontroller, driving a physical 7-segment display. You'll interact with it via the serial monitor, sending digits (0–9) to be decoded by your neural net in real time.

✅ Step-by-step Tasks

🔁 Step 1: Port Forward-Only Neural Net to ESP32

Port the Code of Part 5 to run on an ESP32. You can use PlattformIO or Arduino Studio.
Extend the code to:

- Wait for a digit to be sent via serial.
- Convert the digit (ASCII '0'–'9') into a 4-bit binary input
    - Feed it into the neural network.
    - Get 7 outputs (a–g).
    - Light up the segments accordingly.

💡 Step 2: Build 7-Segment Display Circuit

Wiring Schematic

- Map Segments a–g → GPIOs
- Resistors: Add ~330Ω series resistors between GPIO pins and each segment

Segment	ESP32 GPIO
A	0
B	4
C	16
D	17
E	5
F	18
G	19

🧪 Step 4: Test the Network on ESP32

- Upload your code.
- Open the Serial Monitor
- Type a digit (0–9).
- Observe the correct segments lighting up on the display.